

LAB 12: XỬ LÝ FORM VỚI REACT HOOK FORM VÀ VALIDATION VỚI YUP

Giai đoạn 1: Khởi tạo và Thiết lập Môi trường

- **Khởi tạo dự án React:** Thiết lập project mới bằng Vite hoặc Create React App.
- **Cài đặt thư viện:** Cài đặt bộ 3 công cụ cốt lõi gồm react-hook-form, yup và @hookform/resolvers (cần nối để RHF đọc được Yup schema).
- **Tổ chức thư mục:** Phân chia kiến trúc thư mục rõ ràng với components/ (chứa UI) và schemas/ (chứa logic validation).

1. Khởi tạo dự án React với Vite:

Mở Terminal (hoặc Command Prompt) tại thư mục muốn lưu dự án và chạy dòng lệnh sau:

```
npm create vite@latest react-form-tutorial -- --template react
```

Giải thích:

- `npm create vite@latest`: Gọi công cụ Vite để tạo dự án mới nhất.
- `react-form-tutorial`: Tên thư mục dự án sẽ được tạo ra.
- `--template react`: Chỉ định ngay cho Vite rằng chúng ta muốn dùng template React (Javascript thuần, không dùng Typescript để bài thực hành cơ bản dễ hiểu hơn).

Sau khi lệnh chạy xong, di chuyển vào thư mục dự án:

```
cd react-form-tutorial
```

2. Cài đặt các gói thư viện cốt lõi:

Trước khi cài đặt thư viện ngoài, cần cài đặt các gói mặc định của React đã được Vite cấu hình, sau đó cài bộ 3 công cụ quản lý Form:

```
npm install
npm install react-hook-form yup @hookform/resolvers
```

Giải thích:

- `npm install`: Tải về các thư viện mặc định (react, react-dom) dựa trên file package.json.
- `react-hook-form`: Thư viện chính giúp quản lý Form State không cần re-render liên tục (Uncontrolled Components).
- `yup`: Thư viện chuyên dụng để tạo ra các tập luật (schema) kiểm tra dữ liệu đầu vào.

- @hookform/resolvers: Đóng vai trò là "người phiên dịch", giúp React Hook Form hiểu và đồng bộ được với các tập luật do Yup tạo ra.

3. Dọn dẹp và thiết lập cấu trúc thư mục:

Để code gọn gàng và chuẩn mô hình dự án thực tế, chúng ta sẽ tạo các thư mục phân chia chức năng.

Tại thư mục src/, hãy tạo thêm 2 thư mục và các file trống như cấu trúc sau:

```
src/
├── components/
│   └── RegisterForm.jsx (Chứa giao diện Form)
├── schemas/
│   └── authSchema.js (Chứa logic kiểm tra lỗi Yup)
├── App.jsx (File chạy chính)
└── App.css (Có thể xóa nội dung mặc định bên trong)
```

Sau khi tạo file, hãy mở src/App.jsx và thay thế bằng đoạn code đơn giản sau để kiểm tra môi trường:

```
// src/App.jsx
import React from 'react';
// Import sẵn Component (chúng ta sẽ code ở giai đoạn 2)
// import RegisterForm from '../components/RegisterForm';

function App() {
  return (<>
    <h1> Bài thực hành: Quản lý Form chuyên nghiệp</h1>
    <p> Môi trường đã cài đặt thành công! Sẵn sàng cho Giai đoạn 2.</p>
  </>
  );
}

export default App;
```

4. Chạy thử và kiểm tra kết quả:

Tại terminal đang đứng ở thư mục react-form-tutorial, khởi động server phát triển bằng lệnh:

```
npm run dev
```

Kết quả mong đợi:

1. Terminal sẽ hiển thị một đường dẫn, thường là <http://localhost:5173/>.

2. Nhấn Ctrl + Click vào đường dẫn đó (hoặc copy dán lên trình duyệt).
3. Trên trình duyệt, chúng ta sẽ thấy dòng chữ "**Bài thực hành: Quản lý Form chuyên nghiệp**" cùng thông báo sẵn sàng.
4. Để chắc chắn thư viện đã nằm trong dự án, có thể mở file package.json, tìm mục "dependencies", chúng ta sẽ thấy react-hook-form, yup và @hookform/resolvers có mặt ở đó.

Giai đoạn 2: Xây dựng Giao diện tĩnh (UI)

- **Tạo Form Component:** Dựng file RegisterForm.jsx hoặc LoginForm.jsx.
- **Dựng bộ khung JSX:** Khai báo các trường dữ liệu thực tế như Họ tên, Email, Mật khẩu, Xác nhận mật khẩu.
- **Định dạng giao diện:** Áp dụng CSS thuần, Bootstrap hoặc Tailwind CSS để cấu trúc form rõ ràng, dễ nhìn cho người thực hành.

1. Tạo giao diện tĩnh cho RegisterForm:

Mở file src/components/RegisterForm.jsx (đã tạo trống ở giai đoạn 1) và dán đoạn mã sau vào.

Đoạn mã này thuần túy là JSX tạo ra các ô nhập liệu (input) cơ bản:

```
import React from 'react';

export default function RegisterForm() {
  return (
    <div className="form-container">
      <h3>Đăng ký tài khoản</h3>
      <form>
        <div className="form-group">
          <label>Họ và tên</label>
          <input type="text" placeholder="Nhập họ và tên..." />
        </div>

        <div className="form-group">
          <label>Email</label>
          <input type="email" placeholder="Nhập email..." />
        </div>

        <div className="form-group">
          <label>Mật khẩu</label>
          <input type="password" placeholder="Nhập mật khẩu..." />
        </div>
      </form>
    </div>
  );
}
```

```

    </div>

    <div className="form-group">
      <label>Xác nhận mật khẩu</label>
      <input type="password" placeholder="Nhập lại mật khẩu..." />
    </div>

    <button type="submit" className="submit-btn">
      Đăng ký
    </button>
  </form>
</div>
);
}

```

2. Thêm CSS để làm đẹp giao diện:

Để form không bị xô lệch, mở file src/App.css và xóa hết nội dung cũ (nếu có), sau đó dán đoạn mã CSS sau vào để tạo hình khối (card) và căn chỉnh các input:

```

.form-container {
  max-width: 400px;
  margin: 0 auto;
  padding: 20px 30px;
  border: 1px solid #e0e0e0;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.05);
  background-color: #fff;
}
.form-group {
  margin-bottom: 15px;
  text-align: left;
}
.form-group label {
  display: block;
  margin-bottom: 5px;
  font-weight: 600;
  font-size: 14px;
  color: #333;
}
.form-group input {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

```

```
    box-sizing: border-box;
    font-size: 14px;
  }
  .form-group input:focus {
    outline: none;
    border-color: #007bff;
  }
  .submit-btn {
    width: 100%;
    padding: 12px;
    margin-top: 10px;
    background-color: #007bff;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 16px;
    font-weight: bold;
  }
  .submit-btn:hover {
    background-color: #0056b3;
  }
  h3 {
    text-align: center;
    margin-top: 0;
    color: #333;
  }
}
```

3. *Hiển thị Component ra màn hình chính:*

Cuối cùng, mở file src/App.jsx để gọi thẻ <RegisterForm/> vào và kết nối với file CSS.

```
import React from 'react';
import RegisterForm from './components/RegisterForm';
import './App.css';

function App() {
  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif', textAlign: 'center',
      backgroundColor: '#f9f9f9', minHeight: '100vh' }}>
      <h2>Bài thực hành: Quản lý Form chuyên nghiệp</h2>

      { /* Gọi giao diện Form ra màn hình */ }
      <RegisterForm />
    </div>
  );
}
```

```
    </div>
  );
}

export default App;
```

4. Kiểm tra kết quả:

Lưu lại tất cả các file (Ctrl + S). Nếu server chưa chạy, hãy gõ lệnh **npm run dev** ở Terminal.

Khi xem trên trình duyệt, chúng ta sẽ thấy một Form đăng ký nằm căn giữa màn hình, có bóng đổ (shadow) nhẹ, các ô nhập liệu được căn chỉnh thẳng hàng rất ngay ngắn. Lúc này, nếu nhấn nút "Đăng ký", trang web sẽ bị load lại (reload) do đây là hành vi mặc định của form HTML chưa được xử lý bằng React Hook Form.

Giai đoạn 3: Khai báo Logic React Hook Form

- **Khởi tạo Hook:** Sử dụng hook useForm để trích xuất các hàm và thuộc tính cần thiết.
- **Đăng ký Input:** Gắn hàm register vào từng thẻ input để thư viện bắt đầu quản lý state của form mà không cần dùng useState.
- **Bắt sự kiện Submit:** Bọc hàm xử lý dữ liệu cuối cùng bằng hàm handleSubmit của thư viện.

1. Cập nhật mã nguồn RegisterForm.jsx:

Mở file src/components/RegisterForm.jsx, xóa nội dung cũ và dán toàn bộ đoạn mã sau vào. Chú ý các dòng có chú thích để hiểu thư viện đang can thiệp vào đâu:

```
import React from 'react';
import { useForm } from 'react-hook-form';

export default function RegisterForm() {
  const { register, handleSubmit } = useForm();

  const onSubmit = (data) => {
    console.log("Dữ liệu Form lấy được:", data);
    alert("Đã lấy dữ liệu thành công! Hãy nhấn F12 mở tab Console để xem.");
  };

  return (
    <div className="form-container">
      <h3>Đăng ký tài khoản</h3>

      <form onSubmit={handleSubmit(onSubmit)}>
```

```
    <div className="form-group">
      <label>Họ và tên</label>
      <input
        type="text"
        placeholder="Nhập họ và tên..."
        {...register("fullName")}
      />
    </div>

    <div className="form-group">
      <label>Email</label>
      <input
        type="email"
        placeholder="Nhập email..."
        {...register("email")}
      />
    </div>

    <div className="form-group">
      <label>Mật khẩu</label>
      <input
        type="password"
        placeholder="Nhập mật khẩu..."
        {...register("password")}
      />
    </div>

    <div className="form-group">
      <label>Xác nhận mật khẩu</label>
      <input
        type="password"
        placeholder="Nhập lại mật khẩu..."
        {...register("confirmPassword")}
      />
    </div>

    <button type="submit" className="submit-btn">
      Đăng ký
    </button>
  </form>
</div>
);
}
```

2. Giải thích các hàm cốt lõi:

- `useForm()`: Là bộ não của thư viện. Khi gọi hook này, form bắt đầu được theo dõi ngầm.
- `register("tên_trường")`: Thay vì phải tạo `useState` và viết hàm `onChange` cho từng ô nhập liệu, chúng ta chỉ cần rải (spread) `...register("key")` vào thẻ `<input>`. Thư viện sẽ tự động theo dõi giá trị của ô đó.
- `handleSubmit`: Hàm này tự động gọi `e.preventDefault()` để chặn form reload. Sau đó, nó gom toàn bộ dữ liệu từ các thẻ `input` đã đăng ký và truyền vào hàm `onSubmit`.

3. Chạy thử và kiểm tra Console:

1. Quay lại trình duyệt (nếu server chưa chạy thì gõ `npm run dev`).
2. Nhấn phím **F12** (hoặc Chuột phải -> Kiểm tra / Inspect), chọn tab **Console**.
3. Điền bừa một số thông tin vào các ô trên form và nhấn **Đăng ký**.
4. Chúng ta sẽ không thấy trang web bị chớp/tải lại nữa. Đồng thời, dưới tab Console sẽ in ra một object dạng JSON chứa toàn bộ dữ liệu vừa nhập (ví dụ: `{ fullName: "Nguyễn Văn A", email: "a@gmail.com", ... }`).

Lúc này, form đã hoạt động và lấy được dữ liệu, nhưng người dùng có thể bỏ trống hoặc nhập sai định dạng email mà hệ thống vẫn chấp nhận. Đó là lúc chúng ta cần đến công cụ kiểm tra (Validation).

Giai đoạn 4: Xây dựng Schema Validation với Yup

- **Tạo file Schema riêng:** Khởi tạo object `authSchema` bằng Yup để tách biệt logic kiểm tra lỗi khỏi UI.
- **Thiết lập Rule cơ bản:** Áp dụng các phương thức có sẵn của Yup để kiểm tra bắt buộc nhập (`required`), định dạng email (`email`), hoặc giới hạn ký tự (`min/max`).
- **Thiết lập Rule đối chiếu:** Hướng dẫn sử dụng hàm `oneOf` và `ref` của Yup để giải quyết bài toán kiểm tra mật khẩu nhập lại có khớp nhau không.

1. Tạo Schema với Yup:

Mở file `src/schemas/authSchema.js` (file trống đã tạo ở Giai đoạn 1), và dán toàn bộ đoạn mã sau vào:

```
import * as yup from 'yup';

export const authSchema = yup.object({
```



```
// Kiểm tra trường fullName
fullName: yup
  .string()
  .required("Vui lòng nhập họ và tên")
  .min(3, "Họ và tên phải có ít nhất 3 ký tự")
  .max(50, "Họ và tên không được vượt quá 50 ký tự"),

// Kiểm tra trường email
email: yup
  .string()
  .required("Vui lòng nhập email")
  .email("Email không đúng định dạng (vd: a@gmail.com)"),

// Kiểm tra trường password
password: yup
  .string()
  .required("Vui lòng nhập mật khẩu")
  .min(6, "Mật khẩu phải có ít nhất 6 ký tự"),

// Kiểm tra trường confirmPassword
confirmPassword: yup
  .string()
  .required("Vui lòng xác nhận lại mật khẩu")
  .oneOf([yup.ref('password')], "Mật khẩu xác nhận không khớp")
});
```

2. Giải thích các hàm của Yup:

- `yup.object()`: Khai báo rằng dữ liệu Form của chúng ta là một đối tượng (Object), bao gồm nhiều trường bên trong.
- `yup.string()`: Bắt buộc giá trị truyền vào phải là chuỗi văn bản (Text).
- `required("thông_báo")`: Trường bắt buộc không được bỏ trống. Chuỗi bên trong ngoặc chính là thông báo lỗi sẽ hiển thị cho người dùng.
- `min()`, `max()`: Giới hạn độ dài ngắn nhất/dài nhất của chuỗi.
- `email()`: Hàm có sẵn của Yup dùng Regex để kiểm tra cấu trúc email hợp lệ.
- `oneOf([yup.ref('password')])`: Đây là kỹ thuật đối chiếu. Yup sẽ lấy giá trị của trường password (thông qua hàm ref) và so sánh xem trường confirmPassword có trùng khớp hoàn toàn hay không.

3. Lưu ý về đặt tên biến:

Để ý các khóa như fullName, email, password, confirmPassword trong file Schema này không?

Chúng **BẮT BUỘC** phải trùng khớp hoàn toàn với các tên biến đã khai báo trong hàm register("tên_biến") ở Giai đoạn 3 (RegisterForm.jsx). Nếu sai lệch dù chỉ 1 ký tự, hệ thống báo lỗi sẽ không thể tự động nhận diện được.

Lúc này, chúng ta đã có file authSchema.js chứa toàn bộ luật lệ, nhưng Form đăng ký (file RegisterForm.jsx) vẫn chưa hề biết đến sự tồn tại của file này. Chúng ta cần một "cây cầu" nối chúng lại ở giai đoạn tiếp theo.

Giai đoạn 5: Tích hợp Validation và Phản hồi UI

- **Kết nối Resolver:** Nạp cấu hình yupResolver(authSchema) vào tham số của useForm để đồng bộ hai thư viện.
- **Trích xuất lỗi:** Lấy object errors từ thuộc tính formState do React Hook Form cung cấp.
- **Hiển thị thông báo động:** Xây dựng logic render các đoạn text cảnh báo lỗi (thường có màu đỏ) ngay dưới mỗi trường input tương ứng khi người dùng nhập sai.

Để kết nối hai thư viện này, chúng ta cần sử dụng công cụ @hookform/resolvers đã cài đặt từ đầu.

1. Cập nhật mã nguồn RegisterForm.jsx:

Mở lại file src/components/RegisterForm.jsx, xóa toàn bộ nội dung cũ và dán đoạn mã sau vào.

Hãy chú ý các dòng có số (1), (2), (3) ở chú thích để xem sự thay đổi:

```
import React from 'react';
import { useForm } from 'react-hook-form';
import { yupResolver } from '@hookform/resolvers/yup';
import { authSchema } from '../schemas/authSchema';

export default function RegisterForm() {
  const {
    register,
    handleSubmit,
    formState: { errors }
  } = useForm({
    resolver: yupResolver(authSchema)
```

```
});

const onSubmit = (data) => {
  console.log("Dữ liệu Form lấy được:", data);
  alert("Đăng ký thành công!");
};

return (
  <div className="form-container">
    <h3>Đăng ký tài khoản</h3>

    <form onSubmit={handleSubmit(onSubmit)}>

      <div className="form-group">
        <label>Họ và tên</label>
        <input
          type="text"
          placeholder="Nhập họ và tên..."
          {...register("fullName")}
          className={errors.fullName ? "input-error" : ""}
        />
        {errors.fullName && <p className="error-
text">{errors.fullName.message}</p>}
      </div>

      <div className="form-group">
        <label>Email</label>
        <input
          type="email"
          placeholder="Nhập email..."
          {...register("email")}
          className={errors.email ? "input-error" : ""}
        />
        {errors.email && <p className="error-text">{errors.email.message}</p>}
      </div>

      <div className="form-group">
        <label>Mật khẩu</label>
        <input
          type="password"
          placeholder="Nhập mật khẩu..."
          {...register("password")}
          className={errors.password ? "input-error" : ""}
        />
      </div>
    </form>
  </div>
);
```

```

        {errors.password && <p className="error-
text">{errors.password.message}</p>}
      </div>

      <div className="form-group">
        <label>Xác nhận mật khẩu</label>
        <input
          type="password"
          placeholder="Nhập lại mật khẩu..."
          {...register("confirmPassword")}
          className={errors.confirmPassword ? "input-error" : ""}
        />
        {errors.confirmPassword && <p className="error-
text">{errors.confirmPassword.message}</p>}
      </div>

      <button type="submit" className="submit-btn">
        Đăng ký
      </button>
    </form>
  </div>
);
}

```

2. Giải thích cơ chế hoạt động:

- **yupResolver(authSchema):** Chúng ta nạp file Schema vào tham số cấu hình của useForm. Từ giờ, mỗi khi form submit (hoặc input thay đổi), thư viện sẽ chạy bộ luật này.
- **formState: { errors }:** Đây là một object chứa toàn bộ các lỗi hiện có. Nếu người dùng nhập sai, React Hook Form sẽ nhét thông báo lỗi của Yup vào đây.
- **{errors.email && <p>{errors.email.message}</p>}**: Kỹ thuật render có điều kiện. Nếu tồn tại lỗi ở trường email (errors.email là có thật), nó sẽ in ra thẻ <p> chứa dòng thông báo lỗi đã viết bên Yup.
- **className={errors.fullName ? "input-error" : ""}**: Đổi class CSS của ô input thành viền đỏ nếu có lỗi.

3. Thêm CSS cho thông báo lỗi:

Để chữ lỗi có màu đỏ và viền input chuyển đỏ như thiết kế, mở file src/App.css và thêm hai class CSS này vào **cuối file**:

```
/* CSS cho đoạn text báo lỗi */
```

```
.error-text {
  color: #dc3545;
  font-size: 13px;
  margin-top: 5px;
  margin-bottom: 0;
}

/* Đổi viền input thành đỏ khi có lỗi */
.form-group input.input-error {
  border-color: #dc3545;
  background-color: #fff8f8;
}
```

4. Chạy thử nghiệm trên trình duyệt:

Lưu lại tất cả các file và ra trình duyệt kiểm tra:

1. Để trống toàn bộ Form và bấm ngay nút **Đăng ký**. Chúng ta sẽ thấy các dòng thông báo lỗi màu đỏ (như "Vui lòng nhập họ và tên", "Vui lòng nhập email") hiện ra đồng loạt, viền input cũng đổi màu.
2. Gõ thử email sai định dạng (ví dụ gõ chữ abc), thông báo sẽ chuyển thành "Email không đúng định dạng".
3. Gõ mật khẩu 123456 và xác nhận mật khẩu là 123, thông báo "Mật khẩu xác nhận không khớp" sẽ xuất hiện.

Đến đây, chúng ta đã hoàn thiện một quy trình quản lý Form và Validation cơ bản nhưng chuẩn mực thực tế. Nếu Form dài và phức tạp, việc viết lại quá nhiều đoạn mã `errors.fieldName` sẽ khá cồng kềnh. Giai đoạn cuối cùng sẽ giúp giải quyết vấn đề đó.

Giai đoạn 6: Tối ưu hóa và Refactor (Phần Nâng cao)

- **Component hóa Input:** Hướng dẫn kỹ thuật sử dụng `React.forwardRef` để tách các thẻ input ra thành một Reusable Component dùng chung cho toàn bộ dự án.
- **Kiểm soát nút Submit:** Vô hiệu hóa (disable) nút bấm dựa trên trạng thái `isValid` (form đã hợp lệ chưa) hoặc `isSubmitting` (form có đang gửi API không).
- **Xử lý hậu Submit:** Giả lập độ trễ của API bằng `setTimeout` và gọi hàm `reset()` để làm sạch toàn bộ dữ liệu form sau khi đăng ký thành công.

1. Tạo Reusable Input Component:

Tạo một file mới tại `src/components/InputField.jsx`. Đây sẽ là khuôn mẫu cho mọi thẻ input sau này.

Vui lòng copy đoạn mã sau vào file InputField.jsx:

```
import React from 'react';

const InputField = React.forwardRef(({ label, type, placeholder, error, ...rest },
ref) => {
  return (
    <div className="form-group">
      <label>{label}</label>
      <input
        type={type}
        placeholder={placeholder}
        ref={ref}
        className={error ? "input-error" : ""}
        {...rest}
      />
      {error && <p className="error-text">{error.message}</p>}
    </div>
  );
});

InputField.displayName = 'InputField';

export default InputField;
```

Giải thích React.forwardRef: React Hook Form hoạt động bằng cách quản lý ref của các thẻ input (...register("name") thực chất là trả về một ref). Khi bọc input bằng một Component tùy chỉnh, phải dùng forwardRef để cho phép ref xuyên qua Component cha và chạm tới thẻ <input> thật bên trong.

2.Cập nhật lại RegisterForm bằng InputField:

Mở file src/components/RegisterForm.jsx. Chúng ta sẽ import InputField và dọn dẹp lại cấu trúc Form cho gọn gàng.

Vui lòng thay thế toàn bộ nội dung RegisterForm.jsx bằng đoạn mã sau:

```
import React from 'react';
import { useForm } from 'react-hook-form';
import { yupResolver } from '@hookform/resolvers/yup';
import { authSchema } from '../schemas/authSchema';
import InputField from './InputField';

export default function RegisterForm() {
  const {
```

```
register,
handleSubmit,
reset,
formState: { errors, isSubmitting }
} = useForm({
  resolver: yupResolver(authSchema)
});

const onSubmit = async (data) => {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Dữ liệu gửi lên Server:", data);
      alert("Đăng ký thành công!");
      reset();
      resolve();
    }, 2000);
  });
};

return (
  <div className="form-container">
    <h3>Đăng ký tài khoản</h3>

    <form onSubmit={handleSubmit(onSubmit)}>

      <InputField
        label="Họ và tên"
        type="text"
        placeholder="Nhập họ và tên..."
        error={errors.fullName}
        {...register("fullName")}
      />

      <InputField
        label="Email"
        type="email"
        placeholder="Nhập email..."
        error={errors.email}
        {...register("email")}
      />

      <InputField
        label="Mật khẩu"
        type="password"
```

```

        placeholder="Nhập mật khẩu..."
        error={errors.password}
        {...register("password")}
      />

      <InputField
        label="Xác nhận mật khẩu"
        type="password"
        placeholder="Nhập lại mật khẩu..."
        error={errors.confirmPassword}
        {...register("confirmPassword")}
      />

      <button
        type="submit"
        className={`submit-btn ${isSubmitting ? 'loading' : ''}`}
        disabled={isSubmitting}
      >
        {isSubmitting ? 'Đang xử lý...' : 'Đăng ký'}
      </button>

    </form>
  </div>
);
}

```

3. Giải thích xử lý trạng thái Submit:

- **isSubmitting:** Thuộc tính này do React Hook Form cung cấp, tự động chuyển thành true khi hàm onSubmit đang chạy (và chờ Promise hoàn thành), sau đó tự động về false.
- **Disable Button:** Khi isSubmitting là true, ta gán disabled={isSubmitting} cho nút bấm. Điều này ngăn người dùng bấm liên tục vào nút "Đăng ký" gây ra lỗi gửi dữ liệu nhiều lần (spam request).
- **reset():** Gọi hàm này sau khi đăng ký thành công sẽ xóa sạch giao diện form, chuẩn bị cho phiên làm việc mới.

4. Thêm CSS cho nút khi đang Submit (Tùy chọn):

Mở file src/App.css và thêm hiệu ứng nhỏ này để người dùng nhận biết hệ thống đang xử lý:

```
.submit-btn.loading {
```


LAB 12: XỬ LÝ FORM VỚI REACT HOOK FORM VÀ VALIDATION VỚI YUP

```
background-color: #6c757d;  
cursor: not-allowed;  
opacity: 0.8;  
}
```